AgentWorkShop: A Governed Supervisory Execution Layer for Industrial Agent Teams

Anonymous Authors

***Abstract*—Large language model (LLM) agents increasingly generate industrial control changes, yet the surroundings of each proposal—who may write which parameter, which limits bind for the current product and recipe, whether a human approved, whether the device accepted, and whether the process improved—are rebuilt per deployment inside application code. This paper presents AgentWorkShop, a governed supervisory execution layer that turns those concerns into one path for agent teams: each intervention is one governed transaction whose admission, approval, actuation, graded write acknowledgment, three-way setpoint consistency, verdict, and compensation remain separately inspectable evidence states. Team identities reach semantic data-acquisition (DAQ) and write-control (DCW) nodes through explicit capabilities under a fail-safe line master switch; protocol drivers, versioned production objects, plugin points, and a typed SDK form the integration surface, with process models, objectives, and policies external. The evaluation exercises this path through protocol-closure and mechanism-ablation drills, agent-team missions, a deterministic controller loop, four concurrent process missions, and a deployment-scale regression campaign on one live multi-tenant instance (many provisioned lines and channels on a single host). Every dispatched setpoint closes with a graded acknowledgment and—where the endpoint supports readback—a three-way consistency check across the recipe, the device readback, and a mirror sample drawn over the acquisition path; pacing windows remain in force under automation, and each scenario's optimization trajectory is archived with per-step attribution. The result is an inspectable, simulation-tier verification of the governance mechanisms themselves, not an optimization-benefit claim.**

***Index Terms*—Agent teams, human-in-the-loop, industrial informatics, write governance.**

I. Introduction

LARGE language model (LLM) agents are moving from advising industrial operations to acting on them: recent systems generate control updates for industrial cyber-physical systems from multi-modal input [1], pair each factory module with an LLM agent that dispatches skills over a tool protocol [2], explore resource capabilities in multi-agent manufacturing [3], coordinate shop-floor tasks [4], and have been studied as real-time controllers of a robot production line [5]. What these demonstrations leave implicit is everything that surrounds the proposal: who may change which parameter, which limits are active for the current product and recipe, whether a human approved, whether the device accepted the command, and whether the process improved. Because these concerns are normally embedded in application code, they are rebuilt per process, protocol, and agent framework—duplicated drivers, ad-hoc permission checks, logs untied to decisions—and a task state such as COMPLETED can coexist with a missed process objective. The plants differ—injection molding, wastewater treatment (WWTP), annealing, and BOPET (biaxially oriented polyethylene terephthalate) film lines, each coupling its objective to pressure, temperature, dosing, or line speed—but all need an auditable bridge from a team-level task to heterogeneous acquisition and control nodes, while LLM-oriented industrial work to date is mostly generation, diagnosis, and single-domain control (e.g., [6], [7]).

AgentWorkShop addresses this execution gap through member–node bindings (a member being an engine-backed channel identity) and shared line, product, recipe, and run context. The response is organizational rather than algorithmic: each intervention is one governed transaction whose admission, approval, actuation, graded acknowledgment, three-way setpoint consistency, verdict, and compensation are left as separate evidence records. Nodes expose engineering quantities rather than register addresses; one governed member–node binding path serves every client, where application-specific integration would duplicate drivers, permissions, and logging per application. Fast regulation, certified protection, and the choice of optimization algorithm remain outside the framework.

The paper makes four contributions:

1) a **capability-based industrial execution architecture** connecting agent-team identity, semantic data-acquisition (DAQ) and write-control (DCW) nodes, heterogeneous protocol drivers, and line–product–recipe–run context, with task membership distinct from physical-write authority;
2) a **reusable integration boundary** of self-describing node and driver contracts across six protocol families (five exercised in the evaluation), versioned production objects, plugin extension points, and a typed software development kit (SDK), into which the reference adapter projects scenario blueprints while process models and policies stay external (a second preset is commissioned by configuration alone, Section V-C);
3) an **evidence-preserving governed intervention workflow** separating admission, optional approval, actuation, three-level write acknowledgment with post-write verification, observation, three-way consistency, the caller's verdict, compensation, and backstop actions; and
4) a **layered evaluation** exercising one governed path through protocol closure with mechanism ablation, agent-team missions, a deterministic controller loop,

TABLE I
Closest LLM-based industrial control systems versus AgentWorkShop (venue, LLM role, write path, production-scoped authority, write evidence).

System (venue)	Role of the LLM	Plant write path	Authority & production context	Write evidence & verification
MADE, ICCAD 2026 [1]	multi-modal control-update generation	generated commands + safety layer	HITL verification of outputs; task-level checks	task success over scripted evaluations
Köhle et al., ETFA 2026 [2]	per-module agents, MCP skill dispatch	skill invocation via OPC UA/MQTT	module-scoped skills	task-resolution rate (6-module plant)
Lim & Kovalenko, CASE 2025 [3]	capability exploration across agents	architecture-level	resource-matching scope	architecture evaluation
Zhao et al., AEI 2026 [4]	shop-floor task dispatch	workshop experiments	coordination layer	dispatch/execution performance
Waseem et al., Mach. Learn. Eng. 2026 [5]	real-time controller in the loop	direct loop actuation (robot line)	prompt-level constraints	line KPIs under LLM control
AgentWorkShop	**replaceable strategy slot**	**governed write path: admission, HITL, dispatch, readback**	**member–node bindings; line master switch; product/recipe-scoped limits**	**graded acknowledgment; three-way consistency; attributed journal; compensation; backstop**

concurrent process missions, and a deployment-scale regression campaign on a live multi-tenant instance (Section V), task state and attainment reported separately throughout.

Scope is stated once here and revisited in Section VI: real protocol stacks execute against author-implemented simulators, scripted policies drive the governed tools (two live engine turns reported separately), and fast regulation, certified protection, physical-plant transfer, and optimizer ranking are outside the claims.

II. Related Work

A. LLM Agents in Industrial Control: From Generation to Governance

Agent-based manufacturing has long linked organizational roles to plant capabilities [8]; LLM-oriented industrial work now spans PLC generation and verification [6]. The closest systems, characterized in Table I, target the control action itself: MADE couples multi-modal perception to control generation with a safety layer and HITL verification, scoring its own task success, with no persistent production-scoped write-evidence path reported [1]; Köhle et al. resolve tasks over a six-module smart-factory plant of per-module LLM agents and tool servers—writes carry module-scoped skills, with no versioned production context or write-evidence chain reported [2]; Waseem et al. place pretrained LLMs inside the control loop as real-time controllers of a robotic line [5]—AgentWorkShop occupies a deliberately different layer, above fast regulation, so the roles are complementary; Lim and Kovalenko explore resource capabilities across manufacturing agents [3] and Zhao et al. dispatch shop-floor tasks [4]—both allocate work among agents, whereas this layer governs how an allocated proposal reaches the plant and what evidence it leaves. A recent review frames agentic AI's industrial value as safety-aware, explainable, HITL decision support rather than autonomous control authority [9]; the contribution here is the mechanism set and benchmark that make that positioning concrete, and Table I characterizes each system as reported in its publication.

B. Frameworks, Standards, Run-Time Guarantees, and Automation Practice

Multi-agent frameworks coordinate tasks and tools, and the Model Context Protocol (MCP) and Agent-to-Agent (A2A) specifications standardize tool and inter-agent interfaces [10], [11]; the tool they expose, however, is a bare function signature carrying no line, product, or recipe context, with no record separating admission from execution. ISA-88 and ISA-95 supply complementary models for recipes and operations [12], [13]; AgentWorkShop's versioned recipes and run dispatch instantiate a reduced subset of that stack, and the governed life cycle transposes run-to-run control disciplines [14] onto a supervisory write path. A separate line enforces run-time guarantees on LLM agents—judge and guard agents, rule-based enforcement [15]–[17]; they gate software tool calls, not plant writes, and their policies could compile onto the enforcement substrate contributed here.

Permissions, range checks, approvals, and audit trails long existed in distributed-control-system (DCS) and SCADA practice—versioned batch recipes and electronic batch journals in batch-control systems—and IEC 62443 codifies their security counterpart [18], while formal shielding, control-barrier functions, and simplex run-time assurance offer stronger specified properties [19]–[21]. Against that practice the differences are substantive: the proposer is a machine-generated agent identity rather than a presupposed human operator, so admission binds to member authority and prompt content stays inert, and is re-evaluated per request against live product and recipe state through versioned objects rather than at role-assignment time; the governed path spans five exercised protocol families above any single control system; and the evidence states—graded acknowledgment, three-way consistency, verdict, compensation—survive strategy replacement, where the three-way check is a supervised special case of classical data reconciliation and gross-error detection [22], applied across recipe, device, and mirrored views of one setpoint rather than across plant balance equations. Neither the six systems above nor a classical chain provides that combination as a unit. AgentWorkShop implements source-dependent checks and a rule-based

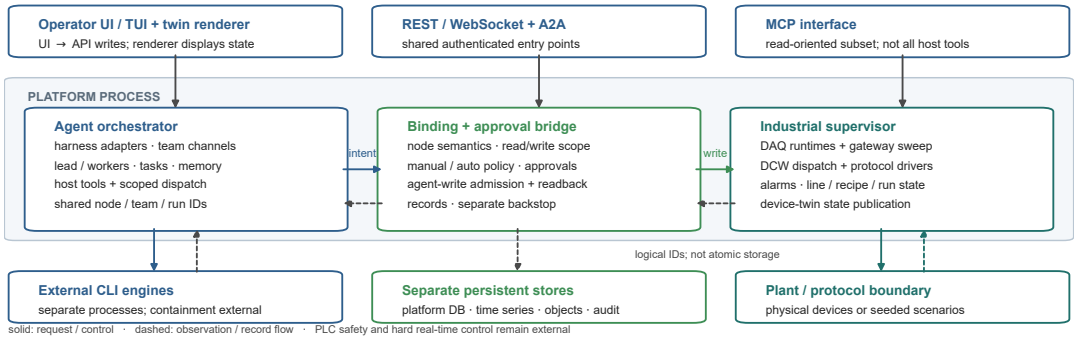


Fig. 1. System architecture and execution boundaries: platform services around shared stores, operator interfaces above, and agent engines, data stores, protocol endpoints, and the plant kept separate. Shared identifiers provide attribution but do not make storage and actuation transactional.

backstop and makes no formal protection guarantee, the backstop (Section IV-D) being a rule-based recovery action in the spirit of ISA-18.2 and EEMUA-191 [23], [24]; human oversight likewise requires more than an approval dialog [25]. Agent benchmarks [26] and prompt-injection studies [27], [28] inform adjacent evaluations; ours separates source checks, protocol execution, mechanism ablation, controller trials, and multi-scenario missions. In sum: coordination frameworks supply no production context, standards no execution path, guard agents gate software tools, and nearest systems stop at the control action—the open slot is the governed, plant-facing write path itself.

III. System Architecture

AgentWorkShop links team coordination to plant interaction through industrial nodes and production context (Fig. 1). Five planes separate the concerns: a team plane owns member identity, task state, and a member-written episodic memory retrieved by hybrid search; an integration plane owns protocol acceptance, readback, freshness, and transport errors, presenting devices as semantic nodes whose stream feeds both the agent and a live digital-twin view [29]; a governance plane owns admission, approval, actuation, acknowledgment grading, three-way consistency, verdict, compensation, and rollback; an operations plane attaches line, product, recipe, and run context while PLC protection stays independent; and an extensibility plane admits plugins, drivers, agent tools, panels, and SDK clients through sanctioned registration services. Scenario knowledge sits in blueprints, signal maps, predicates, and driver configuration—not in team scheduler, approval service, or persistence code; members are interchangeable across 14 engine adapters under preserved identity, and 118 hot-reloadable settings plus the typed SDK make team topology, bindings, and limits data rather than code. In every plane the execution path is unchanged—a scenario supplies data and strategy (Fig. 1).

A. Supervisory Core and Nodes

The Node.js core hosts channel routing, industrial services, and protocol adapters. A channel contains a lead, workers, tasks, messages, and retrievable memory, deployed through WebSocket, MCP, A2A, REST, CLI, or TUI entry points; channel membership alone confers no write authority.

DAQ nodes convert device readings to engineering quantities across four modalities—scalars, vector profiles, image frames carrying content fingerprints, and mirrored manufacturing-execution-system (MES) records—so plant data-shape heterogeneity does not reach the team loop. MES endpoints are reached in two interchangeable modes—mirror, pulling MES records through the same acquisition pipeline as any field device, and direct, declaring read and write mappings against the MES interface. DCW nodes translate engineering setpoints into driver commands and, where supported, readback; per-node runtimes manage timing, and gateway services supply connection state, recipe context, and write bookkeeping. The acquisition surface declares six protocol families, five with drivers today—Modbus TCP, Modbus RTU-over-TCP, OPC UA, MQTT, HTTP/REST, and an S7 stub that fails loudly until its driver lands; the write surface is six drivers over the five writable families plus MES REST, with the evaluation's mock driver registered explicitly and outside that count; the evaluation instantiates five families as lines (Section V-C). The deployment reaches author-implemented dynamic simulators through the same node and driver contracts a plant would use; they are deterministic per seed, not validated digital twins. The first reuse boundary is declarative—signals, units, scaling, precision, and driver configuration—and the team loop is unchanged; fast regulation and independent protection stay in plant controllers.

Driver contracts are deliberately narrow: a driver declares how a semantic quantity maps to its transport payload, whether the endpoint is writable, and whether a readback is available, while the controller resolves member authority and active limits. Errors attach to the request rather than one success flag, and register addresses and byte order appear only here, so a new fieldbus family leaves the loop untouched. Undeclared drivers fail loudly: a node whose driver is not registered is rejected at creation and connection test rather than silently served by a mock (Section V-G). Acquisition is supervisory rather than hard real-time: each DAQ node runs its own cadence under a rotating sweep, so a slow endpoint delays only its own

samples, and the verdict stage queries the timestamped, quality-stamped history rather than issuing a blocking read.

B. Bindings, Context, and Reuse Boundaries

A member–node binding grants access to a sanctioned tool on a specific node and selects automatic or approval-mediated execution. Line permissions (none, read-only, operate) authorize binding creation and mutation; runtime invocation is authorized by the persisted capability, not channel membership. Intervals are re-checked every request instead of cached, so revoking a binding or switching the recipe takes effect on the next request; the same path serves a console, a harness, and an agent engine, because authority does not depend on the client—only the evidence record differs.

Production objects are versioned rather than overwritten: a recipe is dispatched against a known-good version, reverted non-destructively, and compared with the values actually applied, and an intervention is read against the definition that was active when it was admitted. Plugins register hooks, routes, agent tools, drivers, and panels only through sanctioned services, so an added driver inherits admission, readback, and evidence capture. Scenario reuse is bounded by an onboarding contract: the engineer declares a blueprint (devices, signals, units, roles, setpoints); a differential "ensure" pass creates missing devices, repairs drift, verifies connectivity, and projects it into semantic nodes and a recipe; a channel is built from the team template, and a mission supplies objective, guards, response windows, and policy. A BOPET worker proposes casting-speed, fast-roll, or rail-width changes through bound DCW nodes; the other three scenarios reuse the same mission pattern with only blueprint, guards, and policy differing (Section V-F); the adapter demonstrates reuse of platform services, not configuration-only commissioning of physical plants. The stores share identifiers rather than a transaction domain: a device may accept a write even if bookkeeping fails, and time-range queries can cross runs unless bounded.

C. Governed Optimization Task Life Cycle

A governed optimization task is a life cycle rather than a single tool call, and AgentWorkShop exposes each transition separately. The task enters a channel carrying goal, line, product, recipe, run, and task identifiers; a lead member claims it and hands it to a worker holding member–node bindings on the relevant nodes, whose fixed job then repeats: read the controlled quantity over a bounded window, propose a setpoint inside the resolved interval, invoke the governed write, wait for the response, and record a verdict (Section V-B instruments it).

Each iteration leaves more than a written value: the write opens an optimization record carrying prior and requested values, the admitting interval, the acting member, and timestamps; the acknowledgment grade and, where supported, the readback close the command path; the journal attributes the change; and attainment is computed against the scenario predicate while the task closes with a workflow terminal state (per-intervention detail in Section IV). This bounds what "automatic optimization" means here: a controller, a scripted policy, a conventional optimizer, or an agent engine may occupy the strategy slot, while authority, evidence, and closure stay on the platform.

IV. Governed Execution Mechanisms

A supervisory intervention must preserve distinctions that an agent tool call usually collapses—permission to propose, admission of a value, driver acceptance, device confirmation, process response, a caller's verdict, and compensation—and AgentWorkShop exposes these as separate evidence states. Three design commitments generate that separation: model output is treated as an untrusted proposal, never as authority; evidence is graded by what is observed rather than by what was claimed; and failure states are surfaced for inspection rather than absorbed into a success flag. Fig. 2 shows the implemented agent-team runtime that carries these steps; the governance and write stages drawn in the figure are deterministic platform logic—no model output can admit, approve, or confirm a write (Stage 1: admission interlock; Stage 2: approval wait). A governed intervention is one transaction in the audit-trail sense: every step leaves its own evidence record, while execution and recovery are deliberately not atomic with device state (Section VI). Six invariants organize the path: authority resolves from the member–node binding, never from channel membership or prompt content; the proposal must lie in the Eq. (1) interval; a manual binding inserts a timed, fail-closed approval wait; the write leaves a graded acknowledgment, not a success flag; three-way setpoint consistency can be checked before any verdict; and compensation is a separately admitted write referencing the record it reverses. Every agent-originated path to the plant passes through the same governed write.

A. Task, Observation, and Admission

The strategy receives a task and goal in production context and decides what to propose and when to stop; the platform governs execution over semantic node parameters—pressure, dosing rate, line speed, rail width—in engineering units.

For an ordinary agent setpoint request on node n, the admission interval is

$$\mathcal{I}_n(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{1}$$

where $\mathcal{H}_n$ is the mandatory hard engineering range (finite and non-empty at node creation) and the remaining terms the baseline, active-product, and active-recipe limits; a missing optional limit contributes no restriction, so the interval is always defined, and t is the admission instant, re-resolved at execution. Should a configuration empty the intersection, every write is rejected with the excluding layer named, so the misconfiguration surfaces at the first

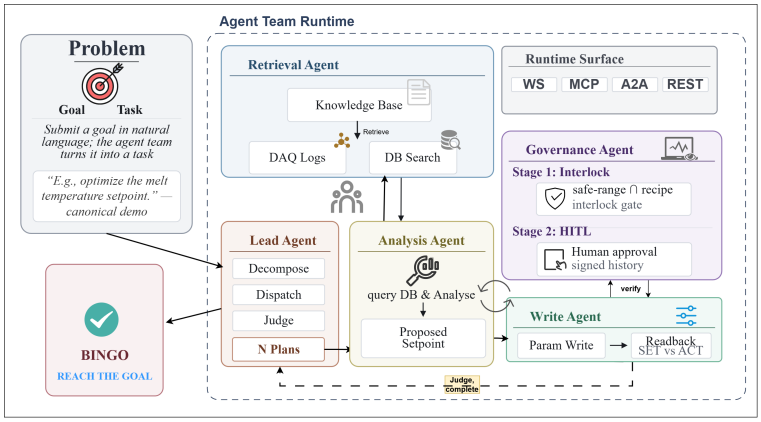


Fig. 2. Implemented agent-team runtime. A natural-language goal enters as a task; the lead agent decomposes and dispatches plans; retrieval and analysis produce the proposed setpoint (the LLM engine occupying a replaceable strategy slot). The governance stages are deterministic platform logic: Stage 1 applies the Eq. (1) interlock, Stage 2 the approval wait; the write agent reads back the value actually present (SET versus observed ACT), and the judge records keep, rollback, or uncertain; every step is journaled with actor attribution. The block labeled "Governance Agent" denotes these deterministic platform stages, not a model-bearing agent; the judge annotation is the caller's verdict-tool invocation.

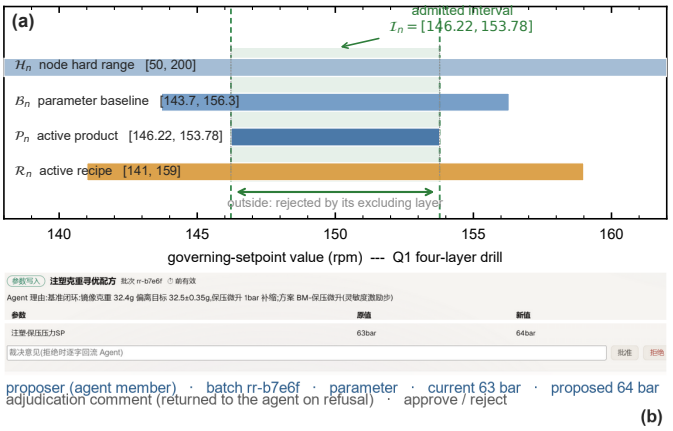


Fig. 3. The admission boundary and its human-approval gate, on the running benchmark line. (a) The Eq. (1) interval geometry: four declared, versioned bounds narrow to the admitted interval; a proposal outside it is rejected by its excluding layer (the Q1 four-layer drill). (b) The structured approval card on a manual binding (live system, gloss added): machine-checkable fields are separated from the proposer's display-only reasoning; the comment returns to the member on refusal (Section IV-E).

write. The interval form is deliberate: every bound is a declared, versioned object that an audit can replay against the active product and recipe, and rejection is decidable and layer-attributable, where a learned feasibility model offers neither (Fig. 3a). It bounds one parameter at a time; joint, cross-variable windows live in scenario guards. It is not a physical safety certificate, and bounds magnitude, not intent: availability, binding state, and approval policy are checked separately, and in-window but adverse sequences surface at judgment and compensation (Section V-F reports two).

B. Semantic Write, Pacing, and Source-Specific Routes

A semantic card exposes quantity, unit, precision, range, value, freshness, and recipe context; the controller resolves binding and limits, the driver serializes the command. The acknowledgment grade, not driver success alone, closes the command path (Section IV-C), never the process-response path.

Two write-spacing windows pace governed loops; neither paces the plant, and the configurable windows are starting points to retune per line. Online writes—agent- or human-origin—must sit at least 60 s apart, and a successful one installs a hold window (writeLockSeconds, floored at 60 s): a further request inside it receives HTTP 429 WRITE_FREQUENT. Recipe dispatch adds a per-recipe online-write interval (opIntervalMs, default 60 s, bounded at 24 h) anchored at the approval instant of the preceding dispatch—pending, refused, or expired requests do not start the clock—keyed by recipe identity and restart-persistent, so alternating agent identities cannot reset it; a coarser per-line trial cadence (default 300 s), also restart-persistent, damps same-line trials from any origin. The hold installs only after driver success, and recipe-dispatch and rollback writes are exempt from both windows. AW_BENCH_MODE=1 admits harness-declared arms that relax the soft window or the readback check, relaxes pacing bench-wide, and never bypasses the hard range.

Four governed routes—agent tools, human-origin writes, recipe dispatch, and rollback compensation—reach the same driver through one admission and journaling entry point, the hard range included; recipe dispatch adds run context—the line-scoped execution window that gates dispatch, the reduced ISA-88 batch/lot notion—validated against the recipe window at save time. Hold heartbeats—the node runtime's periodic re-dispatch of the stored setpoint, off by default, skipped on write conflict, and stopped by line stop, node disable, or driver change—are the one non-governed re-dispatch: the stored value itself changes only through the governed path, but a heartbeat re-applies it without re-admission, so a narrowed recipe window can leave it reasserting a value admitted under the previous one; heartbeats are plant-facing state maintenance, not an agent path. Line start installs active context before recipe parameters dispatch individually, and line stop clears context and closes retained open records; these multi-step

operations are not atomic with device state.

C. Write Acknowledgment as Three-Level Evidence

A driver-level success flag conflates two facts: the transport accepted the command, and the device now holds the value. AgentWorkShop grades every governed write into one of three acknowledgment levels: readback-verified when an independent read—one issued outside the write path—returns the commanded value within the node's configured write tolerance; transport-acknowledged when the transport accepted the command on an endpoint with no usable readback; unverified when the transport failed or when post-write verification could not confirm the value. The grading is fail-honest: absence of evidence degrades the grade, never the report—a write without confirmation is surfaced as unverified, with the transport error attached, rather than dropped from the receipt; the grade is fixed when the write closes, so later confirmation belongs to the record and to the three-way check, not retroactively to the receipt.

A transport-acknowledged write may still not have reached the device, so a post-write verifier re-reads the endpoint (three attempts, 2 s apart) and promotes the grade when a read confirms the value. Batch receipts carry a per-parameter grade plus a summary; the write journal stores the per-write grade, and the operations log records the judgment at worst-state severity. Recipe rollback restores a definition, not physical PLC values, and a compensating write is itself graded by the same three levels. The grade closes the command path only: readback-verified says the device holds the value, not that the process responded as intended—that distinction belongs to observation and verdict.

D. Observation, Judgment, and Compensation

An accepted write opens an optimization record; the verdict tool dcw_judge records keep, rollback, or uncertain plus a reason; the verdict, stored with the record it closes and the response window actually queried, performs no actuation and does not independently validate the evidence, so a scripted keep can coexist with a missed objective—verdict integrity rests on attribution, journaling, and operator review: a kept-but-marked record stays visible with its disagreement attached, and the operations log raises parameter severity under alarm-rationalization practice (Section VI). A three-way consistency test compares the recipe value, the device readback, and a mirror sample drawn from the acquisition runtime's separate polling path and store, each leg against a stated tolerance; the sampled legs meet at the plant endpoint, so a fault fabricating the endpoint is a common-mode boundary. Disagreement marks the record even when every grade verifies. Verdicts are attributable—actor and timestamp travel with them; corrections amend the stored reason visibly, and compensation opens a new record referencing the one it reverses.

Compensation is a new, separately admitted write that may fail or partially complete; its dispatch is graded by the Section IV-C levels, and a rollback record settles to rolled back once the compensating write is accepted—readback-verified where the endpoint supports readback, transport-acknowledged otherwise. If the compensating write is itself refused by admission (a baseline can drift out of a narrowed window), the source record closes as judged, not rolled back, with the refusal appended to its verdict and surfaced in the operations log. These separations keep command status, record status, and process recovery independently inspectable.

The backstop is the platform's safety net for writes that no verdict has closed: it evaluates only open records—a caller's keep or rollback closes the record and stands the backstop down. When the recorded value of an open record leaves the admitting recipe window, the platform issues a system-attributed rollback verdict and restores the record's baseline (the value preceding the write), the compensating write graded as above. The stored policy waits at least 120 s, checks at most every 30 s, and triggers once three out-of-window points or averages accumulate in the observation window—cumulatively, not necessarily consecutively. Points that do not return contribute no breach, so an acquisition outage leaves the record open, monitored but untriggered, a silence the platform does not currently alarm on. Recovery executes only while fewer than two retained rollback records stand on the node—past that, the backstop still judges and records but defers actuation to the approval policy that governed the write, bounding any rollback loop at the two-record cap.

E. Human Approval Boundary and the Line Master Switch

Manual bindings insert the timed approval wait before an agent setpoint or compensation request; automatic bindings omit the wait but keep the checks. The pending request shows member, node, value, interval, and expiry—in the deployed benchmark, plus the proposer's reasoning and per-parameter basis (Fig. 3b): machine-checked fields are structured data, the free-text reasoning display-only and non-authoritative. Because binding and admission are re-checked at execution, approval reserves neither interval nor device state; the approval dialog—itself an injection surface (Section VI)— cannot promote a proposal past the checks, and the comment returns verbatim to the proposing member on refusal and with the receipt on approval—feedback, not authority. The wait fails closed by default (expiry 180 s, configurable): an expired request resolves as refusal, does not actuate, and is recorded with the expiry; duplicates are suppressed by holding at most one pending request per member–node pair regardless of the proposed value, and card creation inherits the write pacing windows, so the pending set is paced like the writes themselves. In hold mode an unanswerable request persists—no expiry, periodic reminders to the line's operate-scoped users—until a human answers; a

repeat governed request for the held node is refused as a duplicate, and other nodes proceed under their own policy.

Human authority is anchored one level above individual requests: each line carries a control mode—the line master switch—whose fail-safe default is manual. Under manual mode, writes from approval-free bindings on that line are intercepted and presented for approval regardless of binding policy, and switching a line to automatic requires an explicit, admin-only user-side confirmation—agent identities hold no binding-administration capability, so an agent cannot reclassify its own writes as approval-free.

The decision endpoint authenticates a user but does not enforce a line-scoped approver role, and reads, verdict recording, and some configuration changes do not use physical-write approval—workflow mediation, not segregation of duties; a recipe switch between approval and execution re-admits the value against the new window (Section VI). The strategy engine is replaceable rather than trusted as a control primitive: the stable plane keeps bindings, semantic I/O, evidence, approval, compensation, and backstop handling, while scenario code declares and configures—never executing plant I/O—and the platform never infers a process objective.

V. Experimental Evaluation

The evaluation asks whether one governed supervisory path can be exercised, inspected, and repeated across heterogeneous industrial jobs, not whether one optimizer beats another. Five questions organize the evidence: protocol closure and admission-gate attribution (Q1), the AgentTeam workflow (Q2), a deterministic controller loop (Q3), four concurrent missions (Q4), and deployment-scale regression with write-evidence verification (Q5); Q2–Q4 occupy the strategy slot (scripted policies, two live engine turns in Q2), Q5 stresses the hardened platform under load, and together the five exercise every plane of Section III.

A. Testbed, Evidence Snapshot, and Scope

The evaluation draws on two evidence tiers of the same codebase. The archived snapshot for Q1–Q4 is integrated run 20260921184204-10lc (seed 42, commit e692df1, harness 9f2756dce4a4cbb3, completed September 21, 2026, 19:08 UTC), archived with raw reports, together with ablation companion 20260921181234-e1lite. The standing single-node preset is a cast-film extrusion line tuned through the die-head heater setpoint (objective J weights thickness conformance, defects, energy, and throughput under melt-temperature and pressure constraints, larger better); each scenario contributes one seed-42 trajectory from initial state: Q1–Q4 are single-trace case studies (each claim they probe is checkable in the archived artifacts), and repeat evidence comes from Q5's consecutive campaigns on the deployment tier—a different tier, not a re-run of these traces.

The deployment-scale tier for Q5 is a live v0.7.57 instance carrying 34 provisioned lines (17 running), 112 active agent channels, 67 online DAQ nodes, 77 write nodes, 52 recipes, and 385 member–node bindings—simulator- and bench-backed objects, not physical plants; multi-tenant means many provisioned lines and channels on one host. The benchmark line is a physics-based injection-molding cell (16 DAQ nodes over the four modalities, 6 control nodes over five protocol families plus MES); all Q5 numbers come from committed per-run reports, ledgers, and timelines (October 7–8: hardened-pipeline run -20261008045434 plus three consecutive full-suite runs; October 9: benchmark-20261009021149/-024141/-033810); the Q1–Q4 snapshot is commit e692df1 of the same code line; its Section IV mechanisms are unchanged except the acknowledgment grading and the master switch, both added by the October hardening pass; the campaign ran under live load.

B. Test Procedure and Campaign Overview

The Q1–Q4 procedure is a 20-phase integrated suite—an author-defined regression instrument, not an external certification—probing one claim per phase, from preset loading against the offline optimum ($J^* = 89.894$) through protocol interception and the record, recipe, approval, and parameter-layer drills (Sections V-C and V-F).

The Q5 procedure is the one-command benchmark pipeline: eight stages (S0 pre-check; S1 API surface matrix, a positive-and-negative sweep of authentication, users, channels, tasks, messaging, permission isolation, both node families, agent tools, HITL, the operations and audit logs, memory, plugins, teams, and notifications; S2 multimodal ingestion with MES dual-mode; S3 governance negative paths; S4 closed-loop optimization; S5 fine-tuning; S6 diagnostics; S7 report) under a harness that captures every check and a timestamped event timeline per run, so each stage exercises one functional surface of Section III and IV end to end. The profile has since grown—S1 adding lockout after repeated failed logins, S4 adding the batch acknowledgment-grade summary—and the October 9 campaign re-runs it three times on the grown profile. No prior system in Table I owns a governed write path to plant endpoints, so the comparator is internal—the ablation arms of Section V-C and mechanism-level suites; no optimization-superiority claim is made. Fig. 4 summarizes the closed-loop instrument.

C. Q1: Protocol Closure and Admission-Gate Attribution

On the five protocol lines the suite stores 55 DAQ samples through real drivers and reads governed writes back on Modbus TCP, OPC UA, MQTT, and HTTP (medians 47.8/19.8/19.3/32.9 ms; the RTU-over-TCP satellite is acquisition-only), with one admission, record, and verdict path serving every request. Each writable line rejects 6/6 out-of-window probes while accepting its legal write (24/24 intercepted); the window-interlock drill probes the window edge and both admission branches. Two drills separate truth paths one success flag would collapse: a frozen-measurement drill (write and readback consistent, measurement out of window) ends with a system-judged

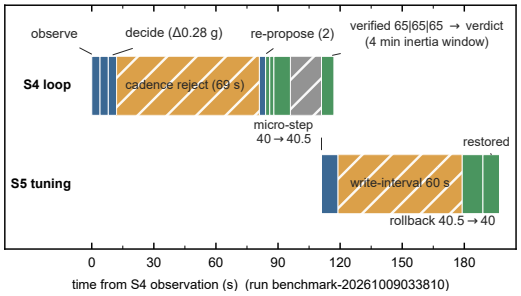


Fig. 4. One corrective governed iteration (harness-adjudicated approval): the trial cadence early-rejects the proposal 69 s before a human is involved; the re-proposal is raised as a HITL card and released by structured approval (green segments: approval, dispatch, verification); dispatch returns readback-verified and the three-way check reads 65|65|65. Lower lane: fine-tuning under the same 60 s window.

backstop rollback 130.2 s later, and a record-rollback drill restores the prior value by a separately admitted, readback-confirmed write. A second preset with different signals and dynamics is commissioned by configuration alone; across both presets 33 probes are rejected on seven writable lines, the same admission code discriminating on unseen signal maps. The parameter layer makes the interval geometry of Eq. (1) observable (Fig. 3a): a four-layer chain—node $[50, 200]$, parameter $[143.7, 156.3]$, product $[146.22, 153.78]$, and recipe $[141, 159]$ rpm—narrowing to one admitted interval, each rejection attributable to its excluding layer; a manual binding suspends a request visibly and unblocks it on approval (latency 461 ms from card raised to approval decision, harness-adjudicated and dominated by the harness's decision poll; Q5 reports the server-side span alone).

The ablation attributes the rejection on companion run 20260921181234-e1lite and a standalone admission fixture (18 attack probes per arm over three repetitions—six per repetition: four hard-range, two window-class—no false blocks, overlapping latencies). Full governance and the no-readback arm reject 18/18; without the window interlock (alone or with the readback check) only 12/18 reject and the six window-class probes execute and are journaled—prevention is the interlock's job; a relaxed configuration degrades from prevention to detection, with the readback check retained to make written-but-unconfirmed commands visible. The hard range is structurally unbypassable and the soft window is its removable part.

D. Q2: The AgentTeam Optimization Workflow and Its Results

The workflow ran end to end on the cast-film line and then on a larger BOPET line (Section III-B); a scripted policy rather than a language model issued the writes (two live engine turns follow). On the single-node mission one write sufficed ($200 \rightarrow 204.704\,^{\circ}$C, settled at $204.70\,^{\circ}$C, journal-attributed, task COMPLETED). The BOPET mission is the multi-node case: nine devices and 49 signals (30 writable, 19 readable), all ensure-pass created. Three governed writes on three distinct nodes—casting speed $32 \rightarrow 33.8$ m/min, fast-roll speed $118 \rightarrow 120.3$ m/min, and rail width $3000 \rightarrow 3035$ mm— bring thickness from 28.10 to 25.66 μm against $25.0 \pm 0.7\,\mu$m; each write opened a record, received keep, and left an agent-attributed journal entry, the parent task reaching COMPLETED, with the governed-write count matching the accepted changes in both missions. Finally, two live agent-engine turns (omp engine adapter, model glm-5.3-flash) drove the same slot on a scratch metal-sheet line—a bounded write (156.3 s) and a goal-driven multi-step descent settling at 51.7 μm against a proposer-chosen target ($52.0 \pm 0.8\,\mu$m, not the scenario predicate), saving a recipe with attributed, audit-logged authorship (637.3 s); the governed path, not an archived predicate, constrains these turns.

E. Q3: The Deterministic Closed Loop

Seeds 42–44 each issue two governed writes with no rejection, raising the cast-film objective J from $J_0 \approx 67.2$ to $J/J^* = 0.965$–0.972, against the offline reference $J^* = 89.894$—an exhaustive grid search over the admissible setpoints (zone temperature 195–225 °C in 2.5 °C steps, screw 50–200 in steps of five, line speed 20–120 in steps of 2.5 m/min) on the same simulator. The gap to unity is expected, as J^* is an oracle with full model knowledge. What the benchmark establishes is narrower: the governed path passes a conventional controller's writes through unmodified—admissible, journaled, read back—transparency of evidence handling, not a performance delta.

F. Q4: Four Process Missions

The campaign runs four blueprint-provisioned lines concurrently; in this run (seed 42) all four attain their archived predicates—evaluated from subsequent observations, not task state. Fig. 5 archives each mission's optimization trajectory. Injection molding is the minimal case (two writes: hold pressure 45→58.2 bar, hold time 6→10.64 s), BOPET the multi-node case of Section V-D; both close with task state and predicate agreeing, on separate records. Wastewater treatment is the write-heaviest mission (15 governed writes): dissolved oxygen rises from 0.21 into 3.4 ± 0.6 mg/L under five effluent guards, with the compliance-cost index down from 181.5 to 179.1; its most informative events are blower steps that were admissible and journaled but lost compliance margin in the post-write observation and were reverted by the verdict stage—a bad proposal is rejected in the evidence, not at the admission gate. Annealing starts far outside the hardness window (134.52 HV, Vickers, at 140 m/min); zone-3 temperature, one zone-2 trim, and two line-speed steps hold 95 ± 6 HV while raising line speed by 17.1%, and a further attempt at 176 m/min is reverted for leaving the margin—on record. Mission clocks (26.48 s, 169.27 s, 102.76 s; BOPET's unarchived) are simulator wall time under accelerated dynamics. Across the four missions in this run, platform-side behavior is identical while the write economy—governed writes per scripted policy—varies by more than a factor of seven (2, 15, 5, and 3).

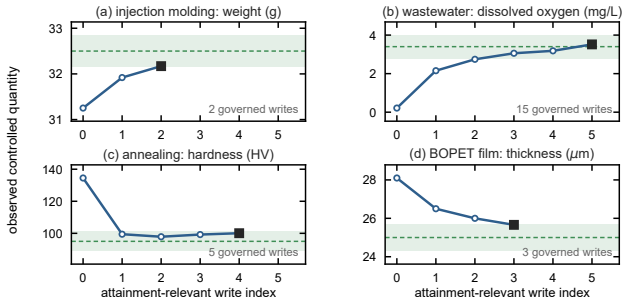


Fig. 5. Archived optimization trajectories of the four concurrent missions (run 20260921184204-10lc, seed 42): observed controlled quantity after each attainment-relevant governed write, against the archived objective window (green band); the square marks the terminal observation. The index counts attainment-relevant writes (6 of 15 plotted for wastewater). Simulator observations under scripted policies.

G. Q5: Deployment-Scale Regression and Write-Evidence Campaign

The final lens stresses the platform itself: the hardening pass made the evidence grading, the line master switch, and the approval-anchored dispatch interval of Section IV the explicit test surface of the closed-loop, fine-tuning, and write-evidence scenarios on the live instance of Section V-A. Full-suite repeatability. Three consecutive pipeline executions complete with zero failed checks; two runs land inside the per-line trial cadence (Section IV-B) and wait out the remaining 218 s— pacing stays in force under automation.

Closed-loop scenario under governance. Each run walks observation, grey-box-sensitized decision (dead-band 0.15 g, slope 0.052 g/bar), a five-element per-parameter proposal, HITL approval, batch dispatch, and three-way consistency (recipe, PLC readback, mirror sample). The loop runs in two regimes: October 7–8 descends 66 → 63 bar one governed step per run under converged dead-band excitation; October 9 answers a sagging mirror weight (32.49/32.33/32.22 g) with a bounded zigzag (64↛63 bar, converged) and corrective steps (63↛65 bar). Every step verifies equal values across the three sources at maximum consistency error 0.0 bar (0.05 bar device, 0.51 bar bucketed-mirror tolerances—the zero is a passed tight check, not equality by construction). The pacing window acts before the human: in two runs the proposal is early-rejected by the trial cadence and re-proposed 69 s later; only then is the HITL card raised (scheme, e.g., hold-pressure micro-up (sensitivity excitation step); proposer reasoning; per-parameter basis) and structured approval releases the readback-verified dispatch (Fig. 4). Approval-to-dispatch completes in 6–9 ms; observation-to-verification spans 25.1 s without a pending cadence window (245.2/245.1 s with, October 7–8; 96.1 s, October 9)—the loop's time is governance by design, not platform overhead. The fine-tuning scenario rides the same windows (Fig. 4, lower lane), and the diagnostics scenario exports a 30-min merged wide table (663–869 rows, 14 columns, both campaigns) with in-specification weight ratio 100%/100%/99.9% (October 7–8; the single outlier a blank boundary cell parsed as zero) and 94.0–100.0% across the three October 9 runs (wider feed spread, std 0.04–0.24 g)—a feed data-quality observation, not a process excursion.

Write-evidence legs. A dedicated seven-leg suite verifies Sections IV-C and IV-E directly: the master switch refuses an unconfirmed manual-to-automatic transition; a mock-acknowledged dispatch is promoted to readback-verified; a broker cut marks affected parameters unverified while healthy siblings verify; a publish-only endpoint stays unverified after three attempts; and on a manual-mode line an approval-free binding's dispatch is intercepted into the queue, approval releasing all six parameters device-verified. An API regression and a documentation-consistency suite (fails on any documented-count or version drift) close the campaign.

VI. Discussion and Limitations

A. Industrial-Informatics Interpretation

One supervisory path carried four different jobs and a deployment-scale campaign with command closure, verdict, task state, and attainment kept as separate records; against the nearest systems of Section II, the differentiator is not the LLM—a replaceable strategy slot—but the production-scoped authority and evidence around its writes, which none of the six systems combines with reversal through one path. Deployment-wise, the layer installs as one service beside the plant's protocol endpoints: a line is onboarded by declaring a blueprint, and approval clients—web panel or API—receive the HITL cards; PLC protection and DCS interlocks stay untouched.

B. Validity and Reproducibility

Run identifier, seed, commit, harness fingerprint, and raw artifacts are fixed, so counts are re-derivable rather than trusted; Q5 adds per-run ledgers and timelines. Five qualifications bound what they support: simulation-tier evidence (deterministic per seed, not validated digital twins; no physical-device campaign, operator study, or external comparison reported); internal-oracle optima and predicates, supporting attribution rather than statistical inference; single-host latencies and near-contemporary runs, bounding repeatability rather than drift; implemented-but-unexercised capabilities and a single live provider/model; and harness-adjudicated approval (human decision latency and quality out of scope).

C. Safety Positioning and Human Oversight

The threat model is a misbehaving or manipulated agent (wrong or hallucinated proposals) inside an otherwise trusted platform and plant; the approval dialog, admission re-checks, and journaling are the containment surfaces, and no adversarial injection campaign is reported (future work); stolen credentials and compromised endpoints are the domain of IEC 62443 [18], [30]. Fast regulation, hardware interlocks, and safety functions remain in the

PLC; prompts and retrieved documents are untrusted text, not authority [27], [28]—the proposer is confined to per-binding scope and journaled, attributable actions: a bounded blast radius, not a protection guarantee. Admission limits, acknowledgment grading, and the backstop run with or without human approval—enforcement never queues behind human attention, while approval fatigue and automation bias remain human-factors risks [25].

D. Deployment Boundaries

Execution and recovery are non-atomic: bookkeeping may fail after actuation, the pacing windows time writes rather than certify stability, readback agreement and three-way consistency need not imply process recovery, and multi-node compensation may partially succeed; enabled hold heartbeats re-apply the stored setpoint without re-admission and can trail a narrowed window; oscillation, storage failure, clock error, and concurrency beyond the four-line campaign remain unevaluated. In production, bindings govern sanctioned tools, not arbitrary privileged-engine I/O—least privilege, endpoint security, and IT/OT separation remain necessary [18], [30]; AgentWorkShop is neither a safety instrumented function nor a plant-protection substitute [31]; line-scoped approver authorization is still missing, so production needs segregation of duties and alarm-path failure tests [24].

VII. Conclusion

AgentWorkShop contributes a governed supervisory execution architecture rather than a new process optimizer: one stable path from agent-team identities through semantic nodes, production context, governed writes, graded acknowledgments, three-way consistency, verdicts, and compensation, exercised by a 20-phase suite, agent-team missions, a three-seed loop, four concurrent missions, and consecutive deployment-scale regression campaigns that complete fully green on a live multi-tenant instance. Physical-plant deployment, operator studies, and external-framework comparison remain open; until then, optimization superiority, autonomous control, and safety assurance stay unsupported claims.

Data Availability and Disclosure

Artifacts are archived per run under bench/results/ and docs/benchmarks/; anonymized artifacts are available during review and released with the code upon publication (harness fingerprints are truncated SHA-256 over benchmark sources). AI assistants supported editing under author direction; the authors remain responsible for all claims.

References

[1] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD), 2026, pp. 1–6.

[2] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA), 2026.

[3] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE), 2025.

[4] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Ji, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," Adv. Eng. Inform., vol. 69, 2026, art. no. 103888.

[5] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," Mach. Learn.: Eng., vol. 2, no. 1, 2026, art. no. 015006.

[6] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," IEEE Trans. Softw. Eng., vol. 52, no. 5, pp. 1672–1687, 2026.

[7] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in Proc. 2025 IEEE 64th Conference on Decision and Control (CDC), 2025, pp. 172–179.

[8] P. Leitão, "Agent-based distributed manufacturing control: A state-of-the-art survey," Eng. Appl. Artif. Intell., vol. 22, no. 7, pp. 979–991, 2009.

[9] X. Jiang, H. Xie, J. Wang, Z. Yang, Y. Zhou, L. Yao, and Z. Zhu, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," Processes, vol. 14, 2026, art. no. 2112.

[10] Anthropic, "Model context protocol," [Online]. Available: https://modelcontextprotocol.io, 2024, accessed: Sep. 20, 2026.

[11] Linux Foundation, "Agent2Agent (A2A) protocol," [Online]. Available: https://a2a-protocol.org, 2025, accessed: Sep. 20, 2026.

[12] International Society of Automation, "ISA-88.00.01-2010—batch control—part 1: Models and terminology," Standard, 2010.

[13] ——, "ANSI/ISA-95.00.01: Enterprise-control system integration," Standard, 2010.

[14] J. Moyne, E. del Castillo, and A. M. Hurwitz, Run-to-Run Control in Semiconductor Manufacturing. Boca Raton, FL: CRC Press, 2000.

[15] Z. Xiang et al., "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," 2024, arXiv preprint arXiv:2406.09187.

[16] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813.

[17] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025, arXiv preprint arXiv:2503.18666.

[18] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2013.

[19] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in Proc. AAAI Conference on Artificial Intelligence, vol. 32, no. 1, 2018.

[20] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," IEEE Trans. Autom. Control, vol. 62, no. 8, pp. 3861–3876, 2017.

[21] L. Sha, "Using simplicity to control complexity," IEEE Softw., vol. 18, no. 4, pp. 20–28, 2001.

[22] C. M. Crowe, "Data reconciliation—progress and challenges," J. Process Control, vol. 6, no. 2–3, pp. 89–98, 1996.

[23] International Society of Automation, "Ansi/ISA-18.2: Management of alarm systems for the process industries (iec 62682)," Standard, 2016.

[24] Engineering Equipment and Materials Users Association, "EEMUA 191: Alarm systems—a guide to design, management and procurement," Guide, Edition 3, 2013.

[25] L. Bainbridge, "Ironies of automation," Automatica, vol. 19, no. 6, pp. 775–779, 1983.

[26] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in Proc. Int. Conf. Learn. Represent. (ICLR), 2024.

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security, 2023, pp. 79–90.

[28] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in Proc. 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, 2024, pp. 1831–1847.

[29] International Organization for Standardization, "ISO 23247: Digital twin framework for manufacturing," Standard, 2021.

[30] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.

[31] International Electrotechnical Commission, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.